HOCHSCHULE ESSLINGEN

OBJEKTORIENTIERTE SYSTEME

INITIALISIERUNG VON OBJEKTEN IN C++

PROF. DR. JÖRG NITZSCHE
HOCHSCHULE ESSLINGEN
JOERG.NITZSCHE@HS-ESSLINGEN.DE

LERNZIEL

- Initialisierung von Objekten durch Konstruktoren vornehmen können
- Verschiedene Konstruktortypen unterscheiden und verwenden können
- Destruktoren korrekt einsetzen können

HOCHSCHULE ESSLINGEN

1 „Initialisierung“ von Objekten
2 Standardkonstruktor
3 Konstruktoren mit Parametern
4 Initialisierungsliste
5 Kopierkonstruktor
6 Konvertierkonstruktor
7 Konstruktoraufruf
8 Destruktoren
9 Standardmethoden und –operatoren

VARIABLEN: DEKLARATION, INITIALISIERUNG UND ZUWEISUNG

- Deklaration
  - Eine Variable wird eingeführt und der Typ der Variable bestimmt, z.B.:
    - `int a; // wie in C`
- Initialisierung
  - Einer Variable wird bei der Deklaration gleich ein Wert zugeordnet, z.B.:
    - `int a = 8; // wie in C`
    - `int a {8};`
    - `int a {};`
    - `auto a = 8;`
    - `auto a {8};`
- Zuweisung
  - Einer Variablen wird NACH der Deklaration ein Wert zugewiesen / der Wert einer Variablen wird geändert, z.B.:
    - `a = 9; // wie in C`

„INITIALISIERUNG“ VON KLASSENELEMENTEN

```
1  #include <iostream>
2  class Fraction {
3      int _numerator, _denominator;
4  public:
5      void init(int n, int d){
6          _numerator  = n;
7          _denominator  = d;
8      }
9      void print(){
10         std::cout <<  "Bruch: " << _numerator << "/" << _denominator  << std::endl;
11     }
12 };
13
14 int main(){
15     Fraction f;
16     f.init(0,1);
17     f.print();
18     return 0;
19 }
```

Ausgabe

```
Bruch = 0/1
```

HOCHSCHULE ESSLINGEN

1 „Initialisierung“ von Objekten
2 Standardkonstruktor
3 Konstruktoren mit Parametern
4 Initialisierungsliste
5 Kopierkonstruktor
6 Konvertierkonstruktor
7 Konstruktoraufruf
8 Destruktoren
9 Standardmethoden und –operatoren

INITIALISIERUNG VON OBJEKTEN

- Ein Objekt wird bei seiner Erzeugung normalerweise nicht initialisiert. Um eine automatische Initialisierung zu bewirken, stehen spezielle Methoden sogenannte **Konstruktoren** zur Verfügung.
- Konstruktoren sind Methoden ohne Rückgabetyp. Der Methodenname eines Konstruktors ist identisch mit dem Namen der Klasse.
- Konstruktoren können nicht explizit aufgerufen werden, sondern der Kompiler benutzt die Konstruktoren, wenn Objekte zu 'konstruieren' sind (Definition, new, ...).
- Der Kompiler stellt Konstruktoren zur Verfügung. Bei der Deklaration einer Klasse kann der Programmierer diese mit eigenen Konstruktoren redefinieren.

STANDARDKONSTRUKTOR DES KOMPILERS

- Der Standardkonstruktor hat keine Parameter.
- Der vom Kompiler zur Verfügung gestellte Standardkonstruktor initialisiert die Datenelemente mit zufälligen Werten (also quasi gar nicht).
- Er heißt auch Defaultkonstruktor, weil er in Fällen 'per default' benutzt wird, wenn es keine Möglichkeit gibt, den Aufruf eines anderen Konstruktors zu veranlassen.

VERWENDUNG EINES OBJEKTS „OHNE INITIALISIERUNG“

```
1  #include <iostream>
2  using namespace std;
3
4  #include <iostream>
5  class Fraction {
6      int _numerator, _denominator;
7  public:
8      void print(){
9          cout <<  "Bruch: " << _numerator << "/" << _denominator  << endl;
10     }
11 };
12
13 int main() {
14     Fraction f;
15     f.print();
16     return 0;
17 }
```

Ausgabe

```
Bruch = -20543360/32767
```

SELBSTDEFINIERTER STANDARDKONSTRUKTOR

```
1  #include <iostream>
2  using namespace std;
3
4  class Fraction {
5      int _numerator, _denominator;
6  public:
7      Fraction() {
8          _numerator   = 0;
9          _denominator  = 1;
10     }
11     void print(){
12         cout << "Bruch: " << _numerator
13              << "/" << _denominator  << endl;
14     }
15 };
16 int main()
17 {
18     Fraction f;
19     f.print();
20     return 0;
21 }
```

Ausgabe

```
Bruch = 0/1
```

- Auch ein selbstdefiniertre Standardkonstruktor hat keine Parameter.
- Der hier gezeigte Standardkonstruktor initialisiert die Attribute `_numerator` und `_denominator` mit den Werten 0 und 1

HOCHSCHULE ESSLINGEN

1 „Initialisierung“ von Objekten
2 Standardkonstruktor
3 Konstruktoren mit Parametern
4 Initialisierungsliste
5 Kopierkonstruktor
6 Konvertierkonstruktor
7 Konstruktoraufruf
8 Destruktoren
9 Standardmethoden und –operatoren

KONSTRUKTOREN MIT PARAMETERN

- Wie eine Methode können Konstruktoren **Parameter** haben, um die Objekte mit individuellen Werten zu initialisieren.
- Bei bestimmten Parameterprofilen haben die Konstruktoren mit Parametern eigene Namen:
  - **Kopierkonstruktor**
    (1 Parameter vom Typ der eigenen Klasse)
  - **Konvertierkonstruktor**
    (1 Parameter von einem anderen Typ)
- Bei den Konstruktoren mit Parametern gelten auch die Regeln für Defaultwerte und für das Overloading.

ÜBERLADEN VON KONSTRUKTOREN

```
1  #include <iostream>
2  using namespace std;
3  class Fraction {
4      int _numerator, _denominator;
5
6  public:
7      Fraction() {
8          _numerator = 0;
9          _denominator = 1;
10     }
11     Fraction(int n) {
12         _numerator  = n;
13         _denominator = 1;
14     }
15     Fraction(int n, int d) {
16         _numerator  = n;
17         _denominator = d;
18     }
19     void print() {
20         cout << "Bruch = " << _numerator
21              << "/" << _denominator << endl;
22     }
23 };
24 int main() {
25     Fraction f_1, f_2(12), f_3(7, -24);
26     f_1.print();
27     f_2.print();
28     f_3.print();
29     return 0;
30 }
```

Ausgabe

```
Bruch = 0/1
Bruch = 12/1
Bruch = 7/-24
```

KONSTRUKTOR MIT DEFAULTWERTEN

```
#include <iostream>
using namespace std;
class Fraction {
    int _numerator, _denominator;

public:
    Fraction(int n = 0, int d = 1) {
        _numerator  = n;
        _denominator = d;
    }
    void print() {
        cout << "Bruch = " << _numerator
             << "/" << _denominator << endl;
    }
};
```

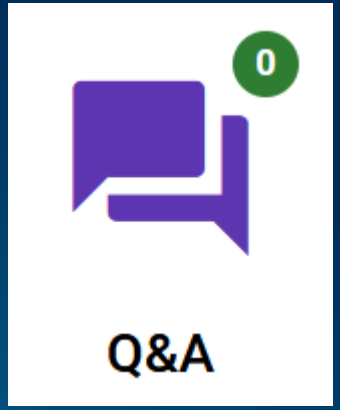
```
int main()
{
    Fraction f_1, f_2(12), f_3(7, -24);
    f_1.print();
    f_2.print();
    f_3.print();
    return 0;
}
```

Ausgabe

```
Bruch = 0/1
Bruch = 12/1
Bruch = 7/-24
```

HOCHSCHULE ESSLINGEN

FRAGEN?



HOCHSCHULE ESSLINGEN

particify.hs-esslingen.de

7751 6136

HOCHSCHULE ESSLINGEN

particify.hs-esslingen.de

7751 6136



Qrticify - QUIZ

```
1  #include <iostream>
2  using namespace std;
3  class Fraction {
4      int _numerator, _denominator;
5
6  public:
7      Fraction() {
8          _numerator = 0;
9          _denominator = 1;
10     }
11     Fraction(int n) {
12         _numerator  = n;
13         _denominator = 1;
14     }
15     Fraction(int n, int d) {
16         _numerator  = n;
17         _denominator = d;
18     }
19 }
```

- Welche Aussagen stimmen:
  - Der erste Konstruktor ist der Standardkonstruktor
  - Der zweite Konstruktor ist ein parametrisierter Konstruktor
  - Der dritte Konstruktor ist ein Konvertierkonstruktor

Articify - QUIZ: AUFLÖSUNG


```
1  #include <iostream>
2  using namespace std;
3  class Fraction {
4      int _numerator, _denominator;
5
6  public:
7      Fraction() {
8          _numerator = 0;
9          _denominator = 1;
10     }
11     Fraction(int n) {
12         _numerator  = n;
13         _denominator = 1;
14     }
15     Fraction(int n, int d) {
16         _numerator  = n;
17         _denominator = d;
18     }
19 }
```

- Welche Aussagen stimmen:
  - Der erste Konstruktor ist der Standardkonstruktor
  - Der zweite Konstruktor ist ein parametrisierter Konstruktor
  - Der dritte Konstruktor ist ein Konvertierkonstruktor

Articify - QUIZ

```
#include <iostream>
using namespace std;
class Fraction {
    int _numerator, _denominator;

public:
    Fraction(int n, int d) {
        _numerator  = n;
        _denominator = d;
    }
    void print() {
        cout << "Bruch = " << _numerator
             << "/" << _denominator << endl;
    }
};
int main()
{
    Fraction f_1, f_2(7, -24);
    f_1.print();
    f_2.print();
    return 0;
}
```

- Was ist das Ergebnis?
  - Bruch = 2456/98302
    Bruch = 7/-24
  - error

Prticify - QUIZ: AUFLÖSUNG


```
1  #include <iostream>
2  using namespace std;
3  class Fraction {
4      int _numerator, _denominator;
5
6  public:
7      Fraction(int n, int d) {
8          _numerator  = n;
9          _denominator = d;
10     }
11     void print() {
12         cout << "Bruch = " << _numerator
13              << "/" << _denominator << endl;
14     }
15 };
16 int main()
17 {
18     Fraction f_1, f_2(7, -24);
19     f_1.print();
20     f_2.print();
21     return 0;
22 }
```

- Was ist das Ergebnis?
  - Bruch = 2456/98302
    Bruch = 7/-24
  - error

Compiler-Ausgabe

```
error: no matching function for call to 'Fraction::Fraction()'
  Fraction f_1, f_2(7, -24);
           ^~~
```

ERLÄUTERUNG ZU FRAGE 2

- Wird ein eigener Konstruktor definiert, so wird der Standardkonstruktor verdeckt.
- Soll er wieder verfügbar gemacht werden, so kann er
  - explizit implementiert werden, z.B.:
    ```
    Fraction(){}
    ```
  - Sichtbar gemacht werden, z.B:
    ```
    Fraction() = default
    ```

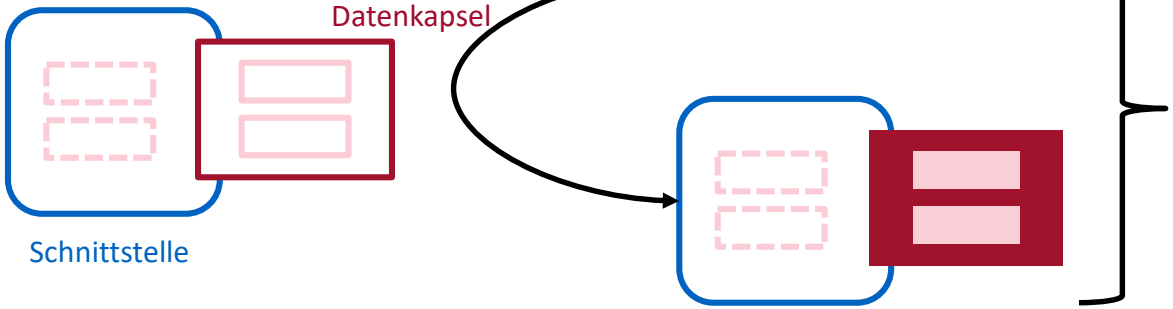
HOCHSCHULE ESSLINGEN

1 „Initialisierung“ von Objekten
2 Standardkonstruktor
3 Konstruktoren mit Parametern
4 Initialisierungsliste
5 Kopierkonstruktor
6 Konvertierkonstruktor
7 Konstruktoraufruf
8 Destruktoren
9 Standardmethoden und –operatoren

INITIALISIERUNG SCHRITTE

- Beim Aufruf des Konstruktors wird zunächst, noch vor Betreten des Blocks, Speicherplatz für die Elemente (im Beispiel `_numerator` und `_denominator`) beschafft.
- Dann wird im zweiten Schritt der Programmcode innerhalb der geschweiften Klammern ausgeführt und die Parameter werden zugewiesen.

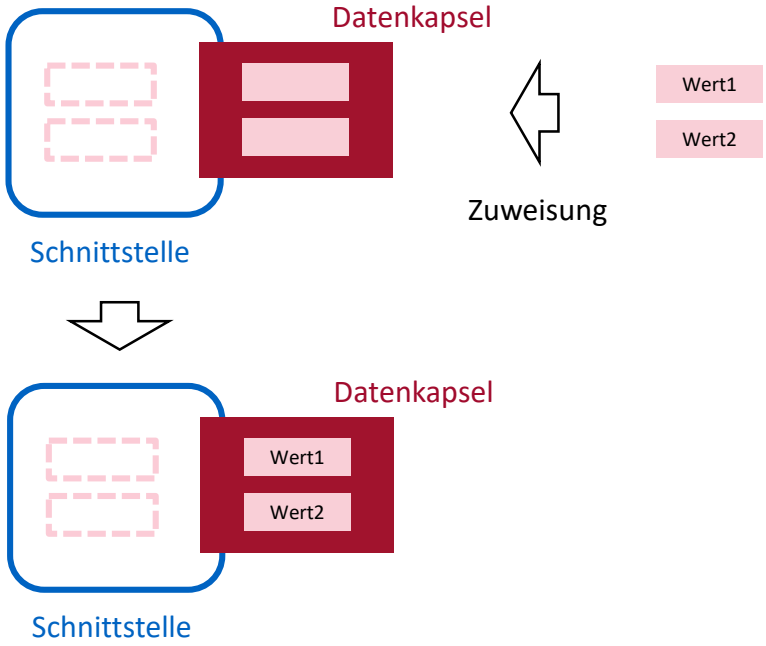
INITIALISIERUNG SCHRITTE – ERSTER SCHRITT




```
#include <iostream>
using namespace std;
class Fraction
{
    int _numerator,
        _denominator;

public:
    Fraction()
    {
        _numerator = 0;
        _denominator = 1;
    }
    ...
};
```

INITIALISIERUNG SCHRITTE – ZWEITER SCHRITT



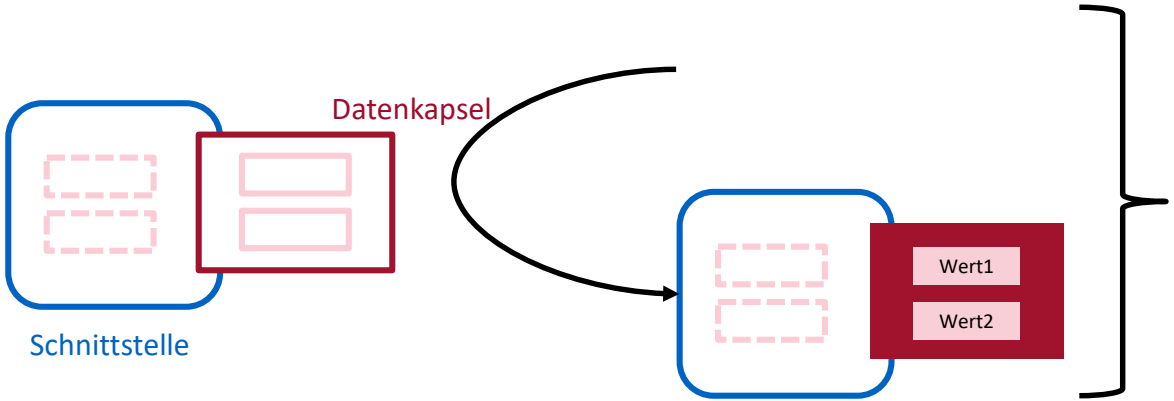

```
#include <iostream>
using namespace std;
class Fraction
{
    int _numerator,
        _denominator;

public:
    Fraction()
    {
        _numerator = 0;
        _denominator = 1;
    }
    ...
};
```

INITIALISIERUNG SCHRITTE

- Es gibt eine Möglichkeit, beide Vorgänge in *einem* Schritt zusammenzufassen, der deshalb besonders bei größeren oder vielen Objekten Laufzeitvorteile bringt.
- Der Weg führt über eine *Element-Initialisierungsliste* (englisch *member initializer list*), die noch vor dem Block angegeben und abgearbeitet wird

INITIALISIERUNG IN EINEM SCHRITT MIT INITIALISIERUNGSLISTE




```
#include <iostream>
using namespace std;
class Fraction
{
    int _numerator,
        _denominator;

public:
    Fraction(int n, int d)
        : _numerator{n},
          _denominator{d}
    {
        // leerer Block
    }
};
```

- Beide Ansätze können auch gemischt werden

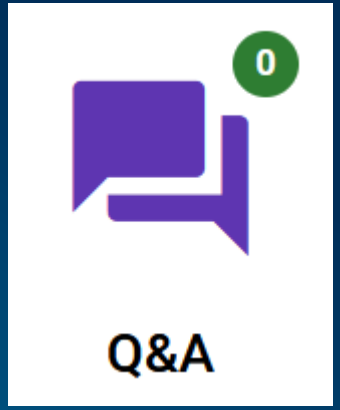
```
1  #include <iostream>
2  using namespace std;
3
4  class Fraction
5  {
6      int _numerator, _denominator;
7
8  public:
9      Fraction(int n, int d)
10         : _numerator{n}
11     {
12         _denominator = n;
13     }
14 };
```

ANWENDUNGSFÄLLE FÜR DIE INITIALISIERUNGSLISTE

- Die Verwendung der Initialisierungsliste ist nicht nur effizienter, sondern es gibt auch Anwendungsfälle für die man die Initialisierungsliste verwenden muss, z.B.:
  - Delegation
  - Aufruf anderer Konstruktoren (siehe Kapitel Vererbung)

HOCHSCHULE ESSLINGEN

FRAGEN?



HOCHSCHULE ESSLINGEN

particify.hs-esslingen.de

7751 6136

HOCHSCHULE ESSLINGEN

particify.hs-esslingen.de

7751 6136



Articify - QUIZ

- Welcher Code führt zu angegebener Ausgabe:

Ausgabe

1 42
2 84

```
#include<iostream>
using namespace std;


// Auswahl an Klassenimplementierungen siehe nächste Folie


int main(){
    Class object_1;
    Class object_2(2,84);
    object_1.print();
    object_2.print();
    return 0;
}
```

Articify - QUIZ

HOCHSCHULE ESSLINGEN

- Welcher Code führt zu angegebener Ausgabe:

Ausgabe

```
1  42
2  84
```

1)

```
class Class {
public:
    Class(int a_1, int a_2): attr_1{a_1}, attr_2{a_2} {}
    Class() : Class(1, 42) {}
    void print(){cout << attr_1 << " "
                      << attr_2 << endl;}
private:
    int attr_1;
    int attr_2;
};
```

2)

```
class Class {
public:
    Class(int a_1, int a_2){attr_1 = a_1; attr_2 = a_2;}
    Class() : Class(1, 42) {}
    void print(){cout << attr_1 << " "
                      << attr_2 << endl;}
private:
    int attr_1;
    int attr_2;
};
```

3)

```
class Class {
public:
    Class(int a_1, int a_2){attr_1 = a_1; attr_2 = a_2;}
    Class(){Class(1, 42);}
    void print(){cout << attr_1 << " "
                      << attr_2 << endl;}
private:
    int attr_1;
    int attr_2;
};
```

4)

```
class Class {
public:
    Class(int a_1, int a_2): attr_1{a_1}, attr_2{a_2} {}
    Class() {Class(1, 42);}
    void print(){cout << attr_1 << " "
                      << attr_2 << endl;}
private:
    int attr_1;
    int attr_2;
};
```

Articify - QUIZ: AUFLÖSUNG

HOCHSCHULE ESSLINGEN

- Welcher Code führt zu angegebener Ausgabe:

Ausgabe

```
1  42
2  84
```

1)

```
class Class {
public:
    Class(int a_1, int a_2): attr_1{a_1}, attr_2{a_2} {}
    Class() : Class(1, 42) {}
    void print(){cout << attr_1 << " "
                      << attr_2 << endl;}
private:
    int attr_1;
    int attr_2;
};
```

2)

```
class Class {
public:
    Class(int a_1, int a_2){attr_1 = a_1; attr_2 = a_2;}
    Class() : Class(1, 42) {}
    void print(){cout << attr_1 << " "
                      << attr_2 << endl;}
private:
    int attr_1;
    int attr_2;
};
```

3)

```
class Class {
public:
    Class(int a_1, int a_2){attr_1 = a_1; attr_2 = a_2;}
    Class(){Class(1, 42);}
    void print(){cout << attr_1 << " "
                      << attr_2 << endl;}
private:
    int attr_1;
    int attr_2;
};
```

4)

```
class Class {
public:
    Class(int a_1, int a_2): attr_1{a_1}, attr_2{a_2} {}
    Class() {Class(1, 42);}
    void print(){cout << attr_1 << " "
                      << attr_2 << endl;}
private:
    int attr_1;
    int attr_2;
};
```

HOCHSCHULE ESSLINGEN

1 „Initialisierung“ von Objekten
2 Standardkonstruktor
3 Konstruktoren mit Parametern
4 Initialisierungsliste
5 Kopierkonstruktor
6 Konvertierkonstruktor
7 Konstruktoraufruf
8 Destruktoren
9 Standardmethoden und –operatoren

KONSTRUKTOREN MIT PARAMETERN

- Wie eine Methode können Konstruktoren **Parameter** haben, um die Objekte mit individuellen Werten zu initialisieren.
- Bei bestimmten Parameterprofilen haben die Konstruktoren mit Parametern eigene Namen:
  - **Kopierkonstruktor**
    (1 Parameter vom Typ der eigenen Klasse)
  - **Konvertierkonstruktor**
    (1 Parameter von einem anderen Typ)
- Bei den Konstruktoren mit Parametern gelten auch die Regeln für Defaultwerte und für das Overloading.

KOPIERKONSTRUKTOR

- Ein **Standard-Kopierkonstruktor** ist für jede Klasse automatisch definiert. Man kann jedoch auch einen Kopierkonstruktor selbst definieren
- Der **Standard-Kopierkonstruktor** kopiert **elementweise**, macht also genau das, was der selbstdefinierte Kopierkonstruktor im Beispiel rechts auch macht.

```
1  #include <iostream>
2  using namespace std;
3  class Fraction {
4      int _numerator, _denominator;
5
6  public:
7      Fraction(int n = 0, int d = 1) {
8          _numerator  = n;
9          _denominator = d;
10     }
11     Fraction(const Fraction& f) {
12         _numerator = f._numerator;
13         _denominator = f._denominator;
14     }
15 };
16 int main()
17 {
18     Fraction f_1;
19     Fraction f_2(f_1);
20     return 0;
21 }
```

- Funktionsdeklaration mit **call-by-value**:

```
int fkt_cbv(Fraction f){
  return f.get_numerator()*f.get_denominator();
}
```

- Zur Herstellung einer **lokalen Kopie** (also zum Initialisieren des Parameters `o` auf dem Stack) wird der **Kopierkonstruktor** aufgerufen.
- Der **Kopierkonstruktor** wird bei jedem Aufruf dieser Funktion verwendet.

REFERENZPARAMETER BEIM KOPIERKONSTRUKTOR

- Zur Vermeidung der Kopie eines aktuellen Parameters kann in C++ ein Referenzparameter benutzt werden (**call-by-reference**):

```
int fkt_cbr(Fraction& f){
  return f.get_numerator()*f.get_denominator();
}
```

HOCHSCHULE ESSLINGEN

particify.hs-esslingen.de

7751 6136



Articify - QUIZ

- Zur Herstellung einer **lokalen Kopie** (also zum Initialisieren des Parameters **`f`** auf dem Stack) wird der **Kopierkonstruktor** aufgerufen.

```
int fkt_cbv(Fraction f){
  return f.get_numerator()*f.get_denominator();
}
```

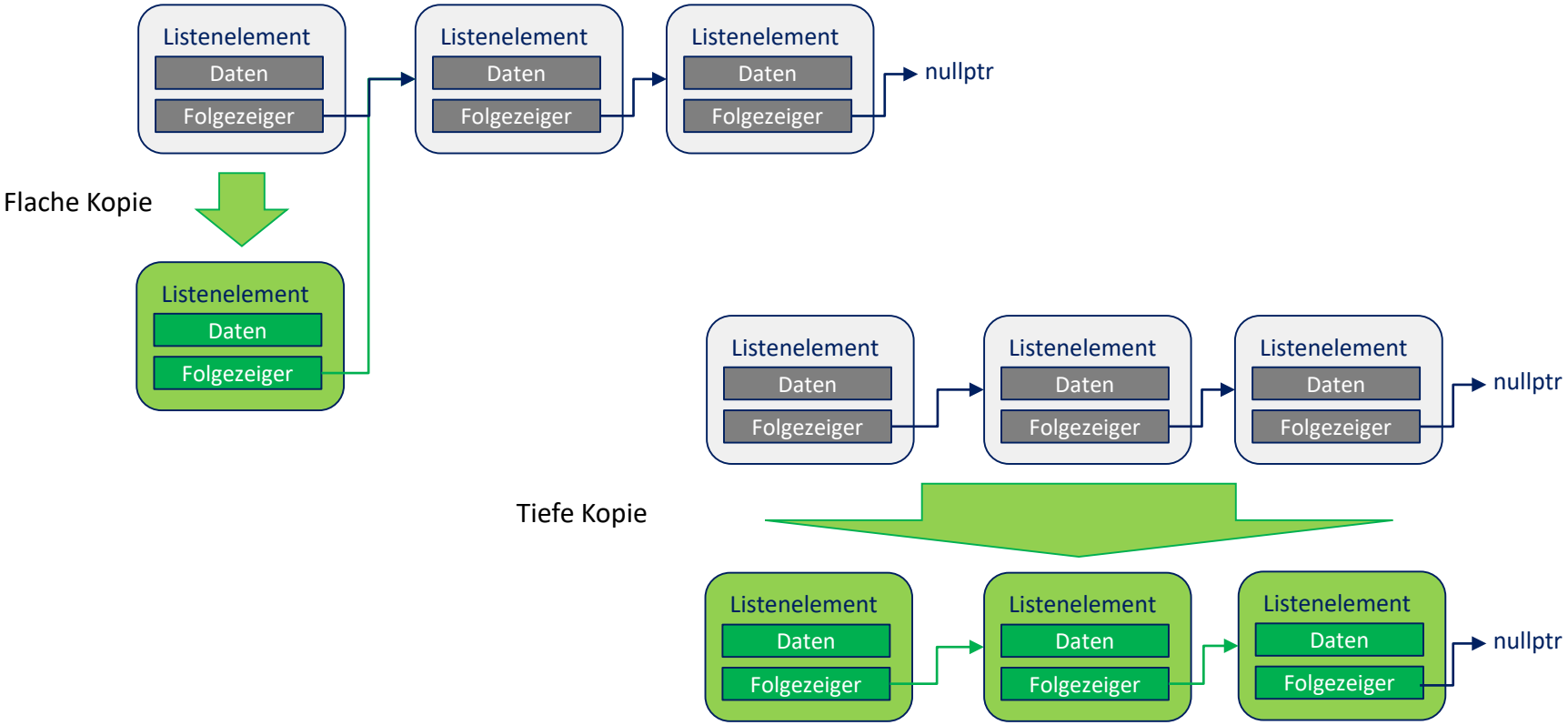
- Was würde passieren, wenn im Kopierkonstruktur auch cbv gemacht würde?

```
Fraction(const Fraction f){
    _numerator = f.get_numerator();
    _denominator = f.get_denominator();
}
```

SELBSTDEFINIERTER KOPIERKONSTRUKTOR

- Wann braucht man einen eigenen Kopierkonstruktor? Häufig dann, wenn dynamische Elemente im Spiel sind (Zeiger)!
- Der Standard-Kopierkonstruktor kopiert nur den/die Pointer, z.B. nur den Anker einer verketteten Liste.
- Man spricht von einer flachen Kopie.
- Will man jedoch die komplette Struktur kopieren, z.B. die ganze verkettete Liste, dann spricht man von einer tiefen Kopie.
- Für eine tiefe Kopie benötigt man einen selbstdefinierten Kopierkonstruktor.

FLACHE KOPIE VS. TIEFE KOPIE



HOCHSCHULE ESSLINGEN

1 „Initialisierung“ von Objekten
2 Standardkonstruktor
3 Konstruktoren mit Parametern
4 Initialisierungsliste
5 Kopierkonstruktor
6 Konvertierkonstruktor
7 Konstruktoraufruf
8 Destruktoren
9 Standardmethoden und –operatoren

KONVERTIERKONSTRUKTOR

- Ein Konvertierkonstruktor ist ein Konstruktor mit einem Parameter, der nicht ein Objekt der Klasse ist.

```
1  class Fraction
2  {
3      int _numerator, _denominator;
4
5  public:
6      Fraction(int n) {
7          _numerator = n;
8          _denominator = 1;
9      }
```

```
10     Fraction(char n) {
12         _numerator = int(n);
13         _denominator = 1;
14     }
15     Fraction(string f) {
16      // string in der Form
17      // "int/int" }
18         ...
```

HOCHSCHULE ESSLINGEN

1 „Initialisierung“ von Objekten
2 Standardkonstruktor
3 Konstruktoren mit Parametern
4 Initialisierungsliste
5 Kopierkonstruktor
6 Konvertierkonstruktor
7 Konstruktoraufruf
8 Destruktoren
9 Standardmethoden und –operatoren

VARIANTEN DES AUFRUFS EINES PARAMETRISIERTEN KONSTRUKTORS

```
1  Fraction fraction_1(70, 90);
2  // Objektdefinition = Konstruktoraufruf (Argumenteliste)
3  Fraction fraction_2 = {70, 90};
4  // Alternative mit Initialisierungsliste {}
5  Fraction fraction_3 {70, 90};
6  // mit Initialisierungsliste {} ohne =
7  Fraction fraction_4 {};
8  // wie {0, 0}
```

VARIANTEN DES AUFRUFS EINES KONVERTIERKONSTRUKTORS

- Folgende Schreibweisen zum Aufruf des Konvertierkonstruktors sind möglich:

```
int i = 5;
Fraction f(i);
Fraction f = i;
Fraction f = (Fraction) i;
Fraction f =  Fraction (i);
Fraction f = static_cast<Fraction>(i);
```

KONVERTIERUNG VON EINER KLASSE IN EINE ANDERE

```
1  class Quotient {
2      double result;
3  public:
4      Quotient (Fraction& f) {
5          result = f.get_numerator()/
6                   f.get_denominator();
7      }
8      void print() {
9          cout << result << endl;
10     }
11 };
12
13 int main()
14 {
15     Fraction f(12, 3);
16     f.print();
17     Quotient q(f);
18     q.print();
19     return 0;
20 }
```

- Der Konvertierkonstruktor kann auch für die Konvertierung einer eigenen Klasse in eine andere genutzt werden:

```
Fraction f(12,3);
Quotient q(f);
```

Ausgabe

```
Quotient = 12/3
4
```

HOCHSCHULE ESSLINGEN

1 „Initialisierung“ von Objekten
2 Standardkonstruktor
3 Konstruktoren mit Parametern
4 Initialisierungsliste
5 Kopierkonstruktor
6 Konvertierkonstruktor
7 Konstruktoraufruf
8 Destruktoren
9 Standardmethoden und –operatoren

DESTRUKTOREN

- Ein **Destruktor** ist das Gegenstück zum Konstruktor.
- Er hat **keine Parameter** und kann **nicht überladen** werden.
- Er wird **automatisch aufgerufen**, wenn der Gültigkeitsbereich eines Objektes verlassen wird, oder `delete` aufgerufen wird.
- Name: `~Fraction();`
- Es gibt einen **Standarddestruktor** des Compilers; er gibt nur den Speicher frei.
- Mit einem **eigens geschriebenen Destruktor** kann man Aktionen veranlassen, die mit dem zu 'destruierenden' Objekt zusammenhängen (Speicher löschen, Datei freigeben, 'tiefe' Struktur löschen).

HOCHSCHULE ESSLINGEN

particify.hs-esslingen.de

7751 6136



Particify - QUIZ

```
#include <iostream>
class  Example
{
public:
     Example(int i = 0) // Konstruktor
        : _number{i}
    {
        std::cout << "Objekt " << _number
                  << " wird erzeugt.\n";
    }
    ~ Example()
    { // Destruktor
        std::cout << "Objekt " << _number
                  << " wird zerstört.\n";
    }
private:
    int _number;
};
```

```
// globale Variable, durch Vorgabewert
// mit 0 initialisiert
Example a_global_example;
int main()
{
    std::cout << "main wird begonnen\n";
    Example an_example(1);
    {
        std::cout << " neuer Block\n ";
        Example an_example(2);
        std::cout << " Block wird
                      verlassen\n ";
    }
    std::cout << "main wird
                  verlassen\n";
}
```

Orticify - QUIZ: AUFLÖSUNG

```cpp
#include <iostream>
class  Example
{
public:
     Example(int i = 0) // Konstruktor
        : _number{i}
    {
        std::cout << "Objekt " << _number
                  << " wird erzeugt.\n";
    }
    ~ Example()
    { // Destruktor
        std::cout << "Objekt " << _number
                  << " wird zerstört.\n";
    }
private:
    int _number;
};
```

```cpp
// globale Variable, durch Vorgabewert
// mit 0 initialisiert
Example a_global_example;
int main()
{
    std::cout << "main wird begonnen\n";
    Example an_example(1);
    {
        std::cout << " neuer Block\n ";
        Example an_example(2);
        std::cout << " Block wird
                      verlassen\n
    }
    std::cout << "main wird
                  verlassen\n";
}
```

Ausgabe

```
Objekt 0 wird erzeugt.
main wird begonnen
Objekt 1 wird erzeugt.
 neuer Block
 Objekt 2 wird erzeugt.
 Block wird verlassen
 Objekt 2 wird zerstört.
main wird verlassen
Objekt 1 wird zerstört.
Objekt 0 wird zerstört.
```

HOCHSCHULE ESSLINGEN

1 „Initialisierung“ von Objekten
2 Standardkonstruktor
3 Konstruktoren mit Parametern
4 Initialisierungsliste
5 Kopierkonstruktor
6 Konvertierkonstruktor
7 Konstruktoraufruf
8 Destruktoren
9 Standardmethoden und –operatoren

STANDARDMETHODEN UND -OPERATOREN

- Standardkonstruktor

```
Fraction();
```

- Defaultkopierkonstruktor

```
Fraction (const Fraction & f);
```

- Defaultdestruktor

```
~Fraction();
```

- Defaultzuweisungsoperator

```
Fraction f_1(1,2), f_2;
f_2 = f_1;
```

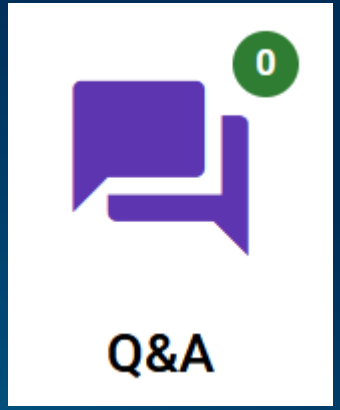
KONSTRUKTOR UND MEHR VORGEBEN ODER VERBIETEN

- Der Compiler erzeugt automatisch einen Konstruktor ohne Parameter, einen Kopierkonstruktor, einen Destruktor und einen Zuweisungsoperator, falls eine Klasse diese nicht zur Verfügung stellt.
- Um dem Programmierer mehr Kontrolle darüber zu erlauben, sind die syntaktischen Konstruktionen = `default` und = `delete` eingeführt worden. Das folgende Beispiel zeigt den Einsatz

```
class X
{
public:
    X() = default; // Compiler-generierten Konstruktor erlauben, obwohl es schon X(int) gibt.
    X(int i);
    X(const X &) = delete; // Kopieren verbieten
};
```

HOCHSCHULE ESSLINGEN

FRAGEN?



HOCHSCHULE ESSLINGEN

particify.hs-esslingen.de

7751 6136

HOCHSCHULE ESSLINGEN

KLEINE ÜBUNG

AUFGABE

- Implementieren Sie ein „dynamisches Array“, das integer-Werte speichern kann. Neue Werte sollen durch die Methode push_back(int) hinten im array eingefügt werden können. Durch die Methode at() sollen die Werte an der entsprechenden Position ausgelesen werden können. Die methode size() gibt die Größe des Arrays zurück. Durch das vorgegebene Hauptprogramm soll die Ausgabe (rechts) erzeugt werden.

Ausgabe

```
Gleich geht es rein...
IntArrayElement erzeugt
IntArrayElement erzeugt
IntArrayElement erzeugt
        size: 3
         0: 10
         1: 20
         2: 30
IntArrayElement zerstört
IntArrayElement zerstört
IntArrayElement zerstört
Jetzt wieder draussen...
```

HOCHSCHULE ESSLINGEN

LABORAUFGABEN

AUFGABE 6

Themen:

Initialisierung von Objekten, Konstruktoren, Destruktoren

Ihre Aufgabe:

Schreiben Sie eine Klasse 'Student' mit den Attributen '_name' und '_first_name' vom Typ String. Schützen Sie die Attribute gegen Zugriff von außen. Die folgenden Methoden implementieren sie außerhalb der Klasse: Die Klasse soll einen parametrisierten Konstruktor haben, der Name und Vorname entgegennimmt und die beiden Attribute '_name' und '_first_name' entsprechend belegt. Verwenden Sie hierzu keine Initialisierungsliste. Schreiben sie get-Methoden für beide Attribute und einen Destruktor. Schreiben Sie zwei print-Methoden, beide ohne Rückgabewert. Eine der beiden Methoden nimmt eine boolsche Variable entgegen, mit der gesteuert werden kann, ob am Ende der Zeile ein Zeilenumbruch erfolgen soll oder nicht. In der anderen Methode erfolgt immer ein Zeilenumbruch.

AUFGABE 6 (2)

Schreiben Sie eine Klasse 'Employee', ebenfalls mit den Attributen '_name' und '_first_name' vom Typ String. Schützen Sie auch diese Attribute gegen einen Zugriff von außen. Die folgenden Methoden implementieren sie innerhalb der Klasse: Die Klasse soll einen parametrisierten Konstruktor haben, der Name und Vorname entgegennimmt und die beiden Attribute '_name' und '_first_name' entsprechend belegt. Verwenden Sie hierzu die Initialisierungsliste. Die Klasse soll einen Standardkonstruktor haben, die den Konstruktoraufruf an den parametrisierten Konstruktor delegiert. Nutzen Sie auch hier die Initialisierungsliste und setzen sie die Werte 'Mustermann' für Name und 'Erika' für Vorname. Schreiben Sie zusätzlich einen Konvertierkonstruktor, der einen Studenten entgegennimmt und die enthaltenen Daten zur Initialisierung von Name und Vorname verwendet. Schreiben sie einen Destruktor. Schreiben Sie auch für diese Klasse zwei print-Methoden, ebenfalls beide ohne Rückgabewert. Eine der beiden Methoden nimmt eine boolsche Variable entgegen, mit der gesteuert werden kann, ob am Ende der Zeile ein Zeilenumbruch erfolgen soll oder nicht. In der anderen Methode erfolgt immer ein Zeilenumbruch.
Die Methoden der beiden Klassen sollen jeweils eine Ausgabe enthalten, die zur Ausgabe der vorgegebenen main-Methode passt. Die Ausgabe ist unten angegeben.

AUFGABE 6 (3)

```
Parametrisierter Konstruktor Student: Max Mustermann
Konvertierungskonstruktor Employee: Max Mustermann
Parametrisierter Konstruktor Employee: Erika Mustermann
Standardkonstruktor Employee: Erika Mustermann
print() ohne Parameter; Student: Max Mustermann
print() mit Parameter; Student: Max Mustermann
print() ohne Parameter; Mitarbeiter: Max Mustermann
print() ohne Parameter; Mitarbeiter: Erika Mustermann
Block wird betreten
Parametrisierter Konstruktor Student: Markus Mustermann
print() mit Parameter; Student: Markus Mustermann
Block wurde verlassen
Destruktor Student: Markus Mustermann
Destruktor Employee: Erika Mustermann
Destruktor Employee: Max Mustermann
Destruktor Student: Max Mustermann
```

HOCHSCHULE ESSLINGEN

ZUSAMMENFASSUNG & AUSBLICK

ZUSAMMENFASSUNG

- Standardkonstruktor
- Konstruktoren mit Parametern
- Initialisierungsliste
- Kopierkonstruktor
- Konvertierkonstruktor
- Konstruktoraufruf
- Destruktoren
- Standardmethoden und –operatoren

WAS KOMMT NÄCHSTE WOCHE?

- Weitere Klassenelemente
- Beziehungen zwischen Objekten

HOCHSCHULE ESSLINGEN

DANKE



HOCHSCHULE ESSLINGEN

particify.hs-esslingen.de

7751 6136